

Operating System

CS 2006

Lecture 15

Mahzaib Younas

Lecturer Department of Computer Science

FAST NUCES CFD

Background

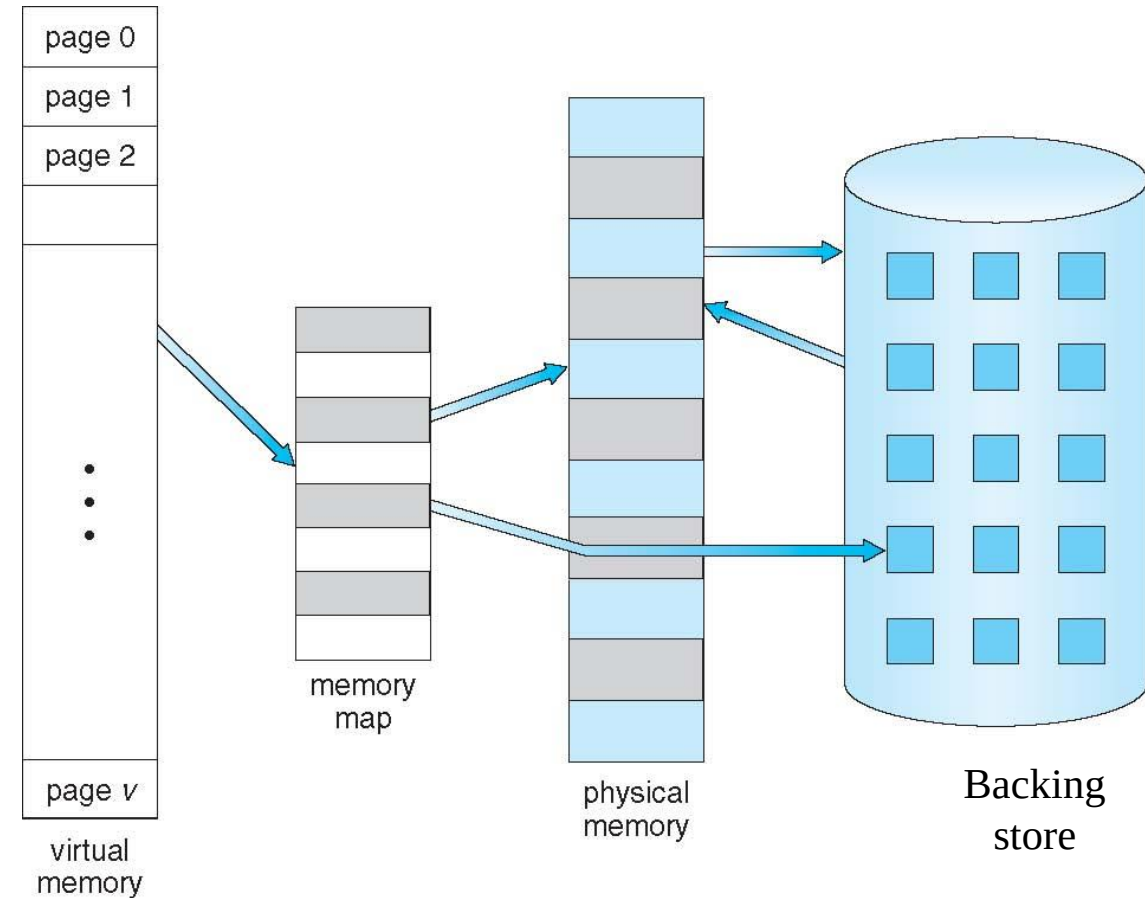
- Virtual memory – separation of user logical memory from physical memory
 - Only part of the program needs to be in memory for execution
 - Logical address space can therefore be much larger than physical address space
 - Allows address spaces to be shared by several processes
 - Allows for more efficient process creation
 - More programs running concurrently
 - Less I/O needed to load or swap processes

Background

- **Virtual address space** — logical view of how process is stored in memory
 - Usually start at **address 0, contiguous addresses until end of space**
 - Meanwhile, **physical memory organized in page frames**
 - MMU must map logical to physical
- Virtual memory can be implemented via:
 - Demand paging
 - Demand segmentation

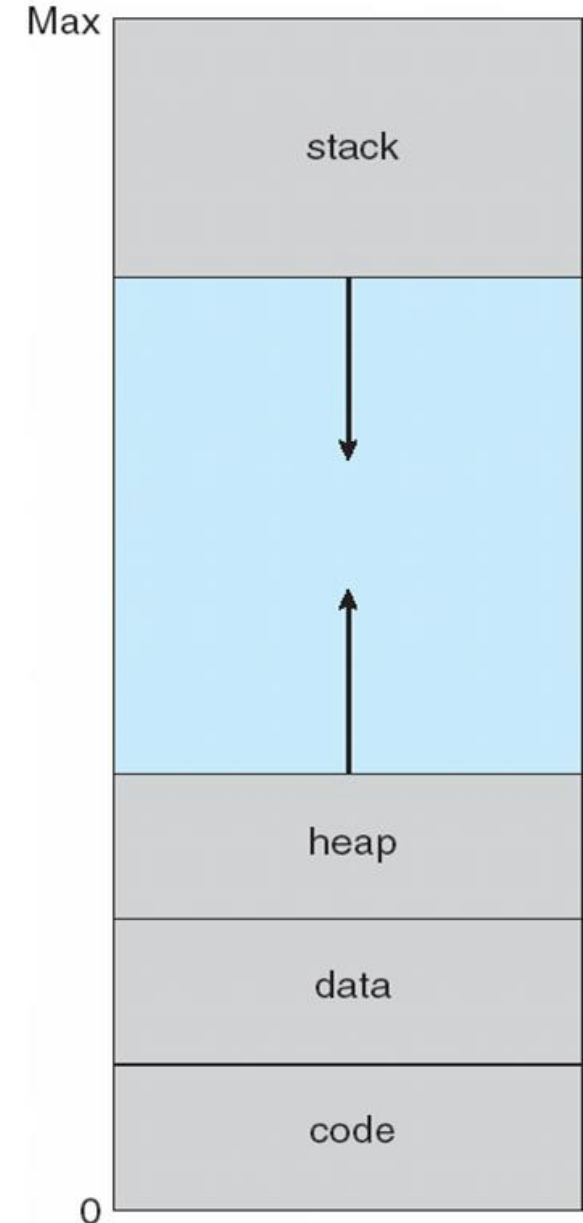
Virtual Memory

- Virtual memory is a memory management technique where secondary memory can be used as if it were a part of the main memory.
- Virtual memory is a common technique used in a computer's operating system (OS).
- This separation allows an extremely large virtual memory to be provided for programmers when only small physical memory is available.



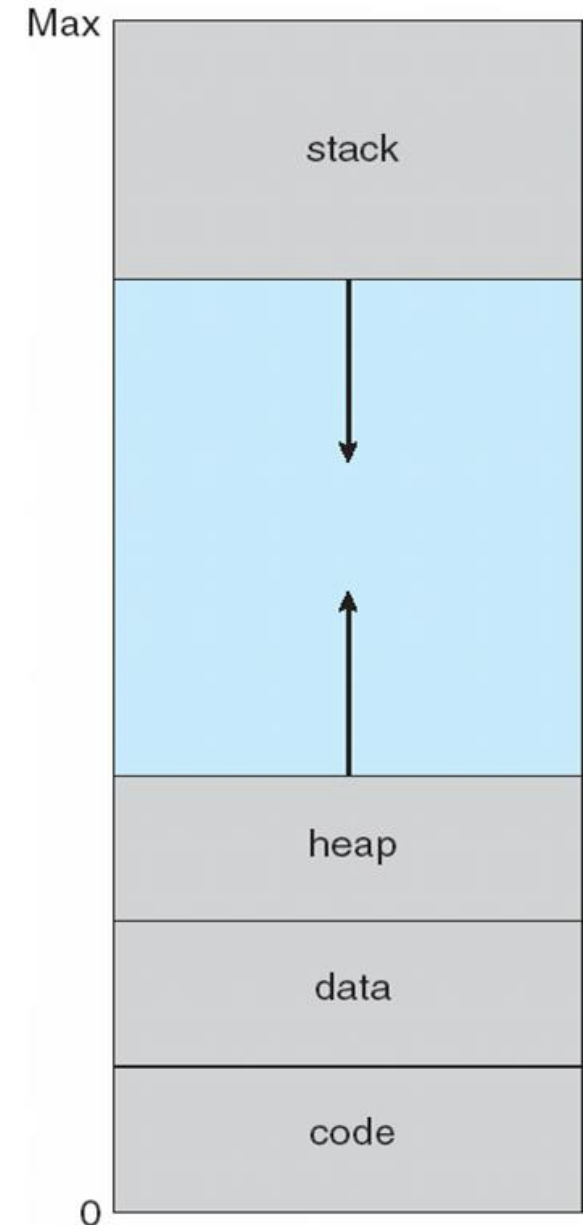
Virtual Address Space

- The virtual address space of a process refers to the logical view of how a process is stored in memory.
- Usually design logical address space for stack to start at **Max logical address** and **grow “down”** while heap grows **“up”**
 - Maximizes address space use
 - **Unused address space between the two is hole**
 - No physical memory needed until heap or stack grows to a given new page

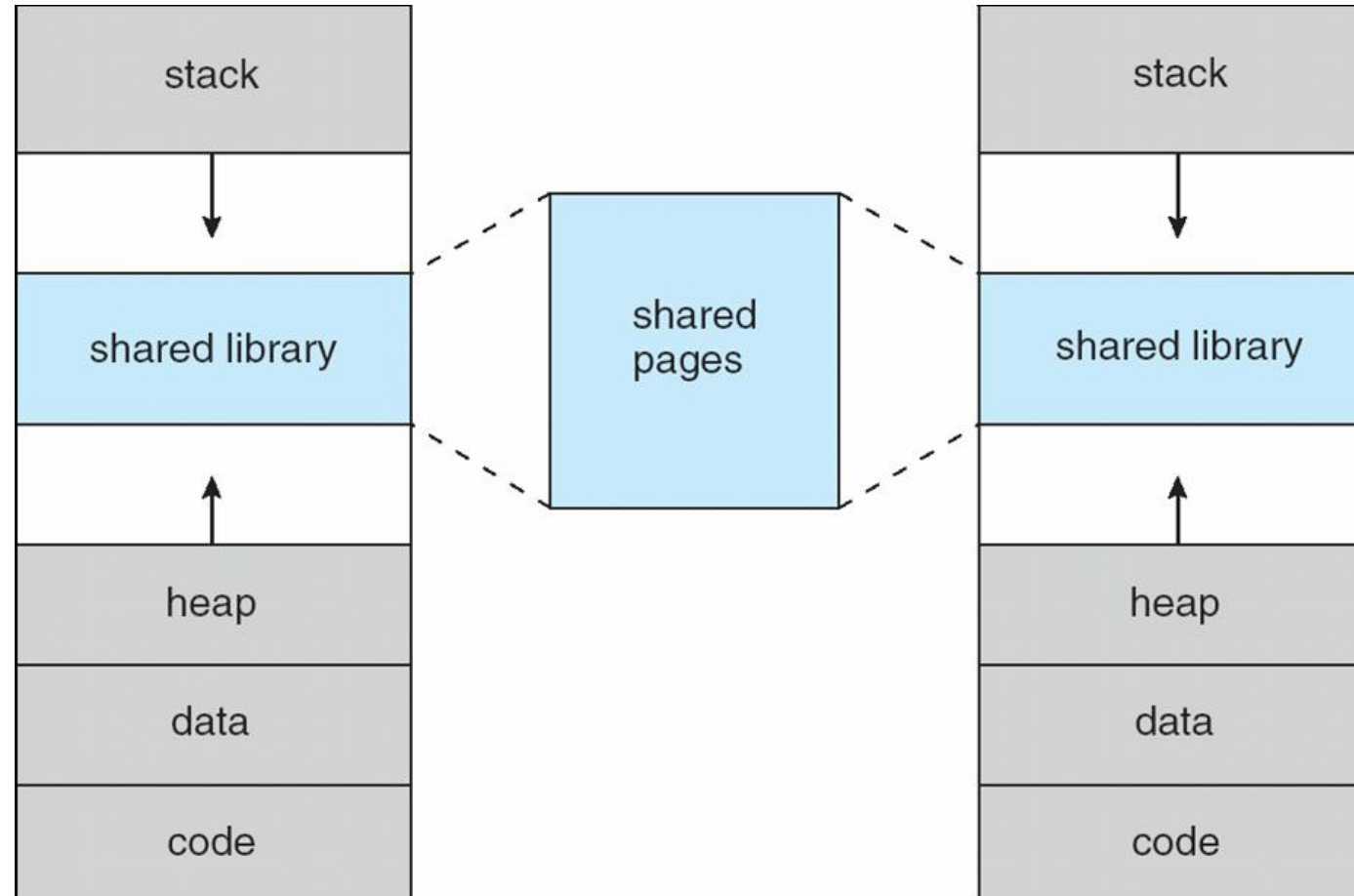


Virtual Address Space

- Enables sparse address spaces with holes left for growth, dynamically linked libraries, etc
- System libraries shared via mapping into virtual address space
- Shared memory by mapping pages read-write into virtual address space
- Pages can be shared during fork(), speeding process creation



Shared Library using Virtual Memory

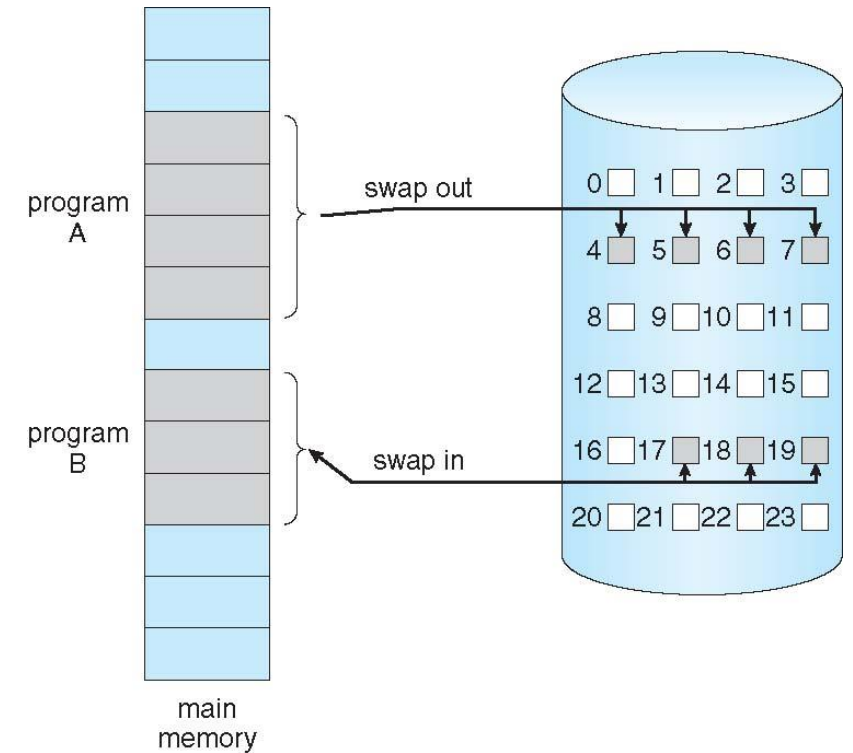


Demand Paging

- Demand paging can be described as a memory management technique that is used in operating systems to improve memory usage and system performance.
- Demand paging is a technique used in virtual memory systems where pages enter main memory only when requested or needed by the CPU.
- Could bring entire process into memory at load time
- Or bring a page into memory only when it is needed
 - Less I/O needed, no unnecessary I/O
 - Less memory needed
 - Faster response
 - More users
- Similar to paging system

Demand Paging

- with swapping (diagram on right)
- Page is needed $\Rightarrow$ reference to it
 - invalid reference $\Rightarrow$ abort
 - not-in-memory $\Rightarrow$ bring to memory
- **Lazy swapper** – never swaps a page into memory unless page will be needed
 - Swapper that deals with pages is a **pager**



Basic Concept

- With swapping, pager guesses which pages will be used before swapping out again
- Instead, pager brings in only those pages into memory
- How to determine that set of pages?
 - Need new MMU functionality to implement demand paging
- **If pages needed are already memory resident**
 - No difference from **non demand-paging**
- **If page needed and not memory resident**
 - Need to detect and load the page into memory from storage
 - Without changing program behavior
 - Without programmer needing to change code

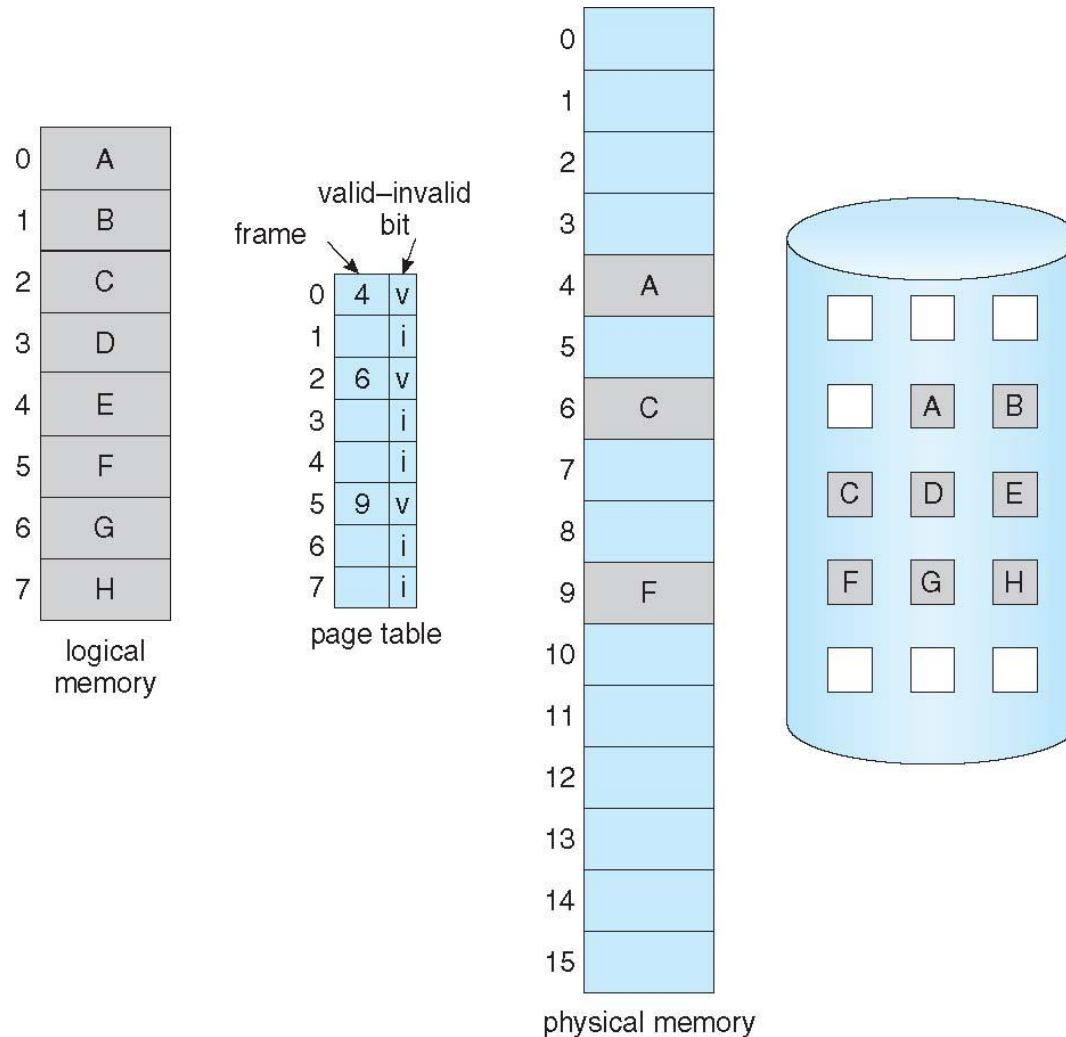
Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated (v = in-memory – memory resident, i = not-in-memory)
- Initially valid–invalid bit is set to i on all entries
- **Example of a page table snapshot:**
 - During MMU address translation, if valid–invalid bit in page table entry is i = page fault

Frame #	valid-invalid bit
	v
	v
	v
	i
...	
	i
	i

page table

Page Table when some pages are not in memory



Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system:

page fault

1. Operating system looks at another table to decide:

- Invalid reference $\Rightarrow$ abort
- Just not in memory

2. Find free frame

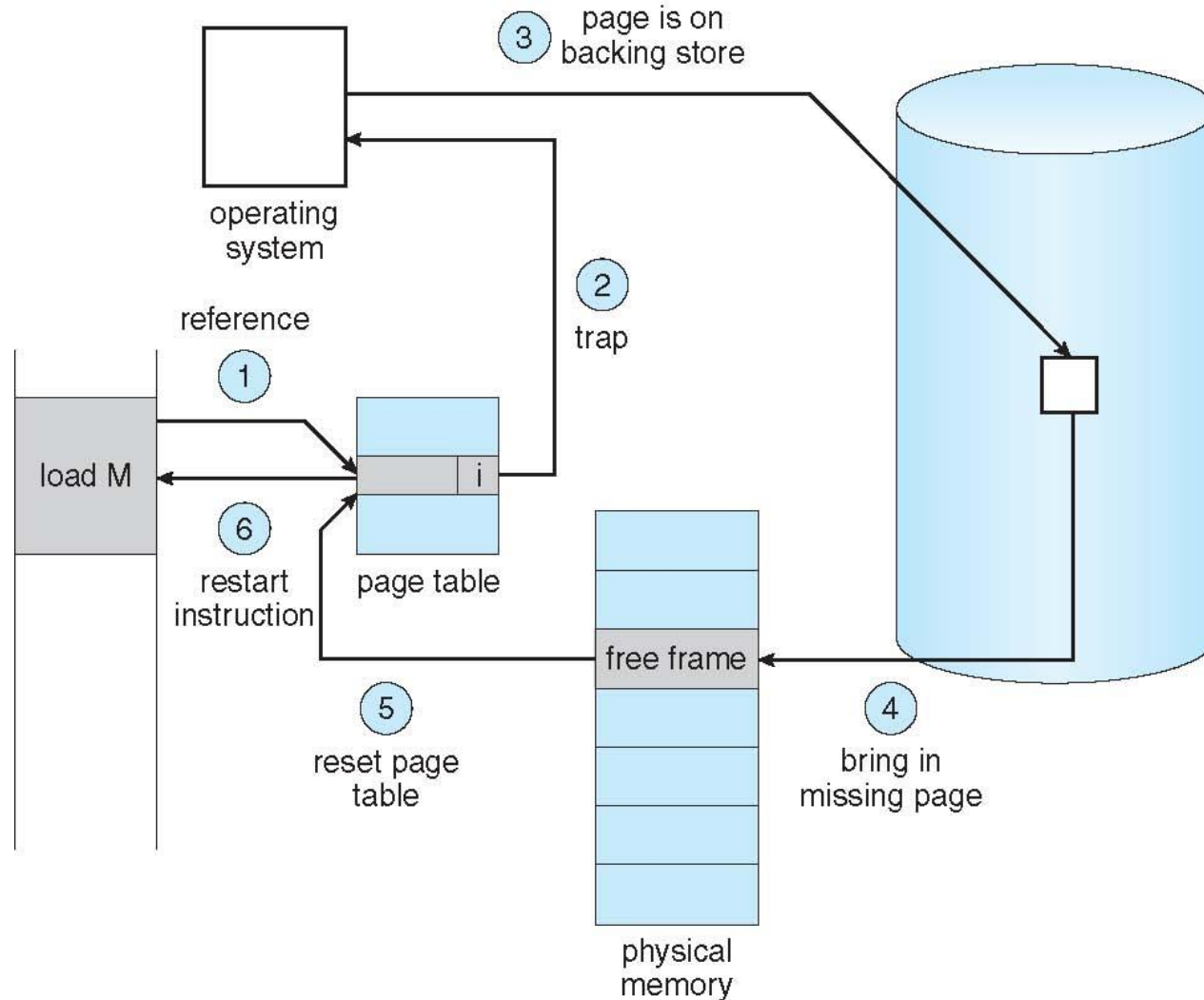
3. Swap page into frame via scheduled disk operation

4. Reset tables to indicate page now in memory

Set validation bit = v

5. Restart the instruction that caused the page fault

Steps in Handling a Page Fault



Aspects of Demand Paging

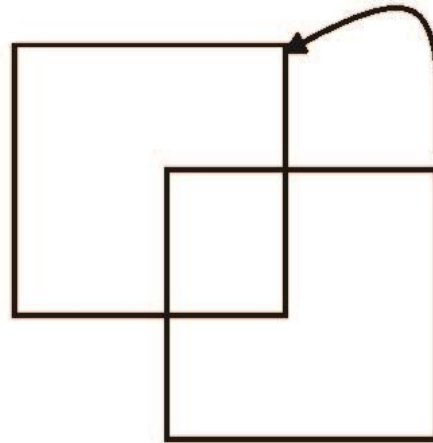
- Extreme case – **start process with *no* pages in memory**
 - OS sets instruction pointer to first instruction of process, non-memory-resident -> page fault
 - And for every other process pages on first access
 - **Pure demand paging**
- Actually, a given instruction **could access multiple pages -> multiple page faults**
 - Consider fetch and decode of instruction which adds 2 numbers from memory and stores result back to memory
 - **Pain decreased because of locality of reference**
- Hardware support needed for demand paging
 - Page table with valid / invalid bit
 - **Secondary memory (swap device with swap space)**
 - Instruction restart

Aspects of Demand Paging

- The hardware to support demand paging is the same as the hardware for paging and swapping:
- Page table.
 - This table has the ability to mark an entry invalid through a valid–invalid bit or a special value of protection bits.
- Secondary memory.
 - This memory holds those pages that are not present in main memory. The secondary memory is usually a high-speed disk or NVM device. It is known as the swap device, and the section of storage used for this purpose is known as swap space.

Instruction Restart

- Consider an instruction that could access several different locations block move



- auto increment/decrement location
- Restart the whole operation?
 - What if source and destination overlap?

Free Frame List

- When a page fault occurs, the operating system must bring the desired page from secondary storage into main memory.
- To resolve page faults, most operating systems maintain a free-frame list, **a pool of free frames**
- **Operating systems typically allocate** free frames using a technique known as zero-fill-on-demand.
- Zero-fill on-demand frames are **“zeroed-out”** before being allocated, thus **erasing their previous contents.**

Performance of Demand Paging

- Stages in Demand Paging (worse case)

1. Trap to the operating system
2. Save the user registers and process state
3. Determine that the interrupt was a page fault
4. Check that the page reference was legal and determine the location of the page on the disk
5. Issue a read from the disk to a free frame:
 1. Wait in a queue for this device until the read request is serviced
 2. Wait for the device seek and/or latency time
 3. Begin the transfer of the page to a free frame

Performance of Demand Paging

6. While waiting, allocate the CPU to some other user
7. Receive an interrupt from the disk I/O subsystem (I/O completed)
8. Save the registers and process state for the other user
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction

Performance of Demand Paging

- Three major activities
 - Service the interrupt – careful coding means just several hundred instructions needed
 - Read the page – lots of time
 - Restart the process – again just a small amount of time
- Page Fault Rate $0 \leq p \leq 1$
 - if $p = 0$ no page faults
 - if $p = 1$, every reference is a fault
- Effective Access Time (EAT)
EAT =
 $(1 - p) \times \text{memory access} + p (\text{page fault overhead} + \text{swap page out} + \text{swap page in})$

EAT Example

- Memory access time = 200 nanoseconds
- Average page-fault service time = 8 milliseconds
- $EAT = (1 - p) \times 200 + p (8 \text{ milliseconds})$
 $= (1 - p) \times 200 + p \times 8,000,000$
 $= 200 + p \times 7,999,800$
- If one access out of 1,000 causes a page fault, then
 $EAT = 8.2 \text{ microseconds.}$

This is a slowdown by a factor of 40!!

- If want performance degradation < 10 percent
 - $220 > 200 + 7,999,800 \times p$
 $20 > 7,999,800 \times p$
 - $p < .0000025$
 - < one page fault in every 400,000 memory accesses

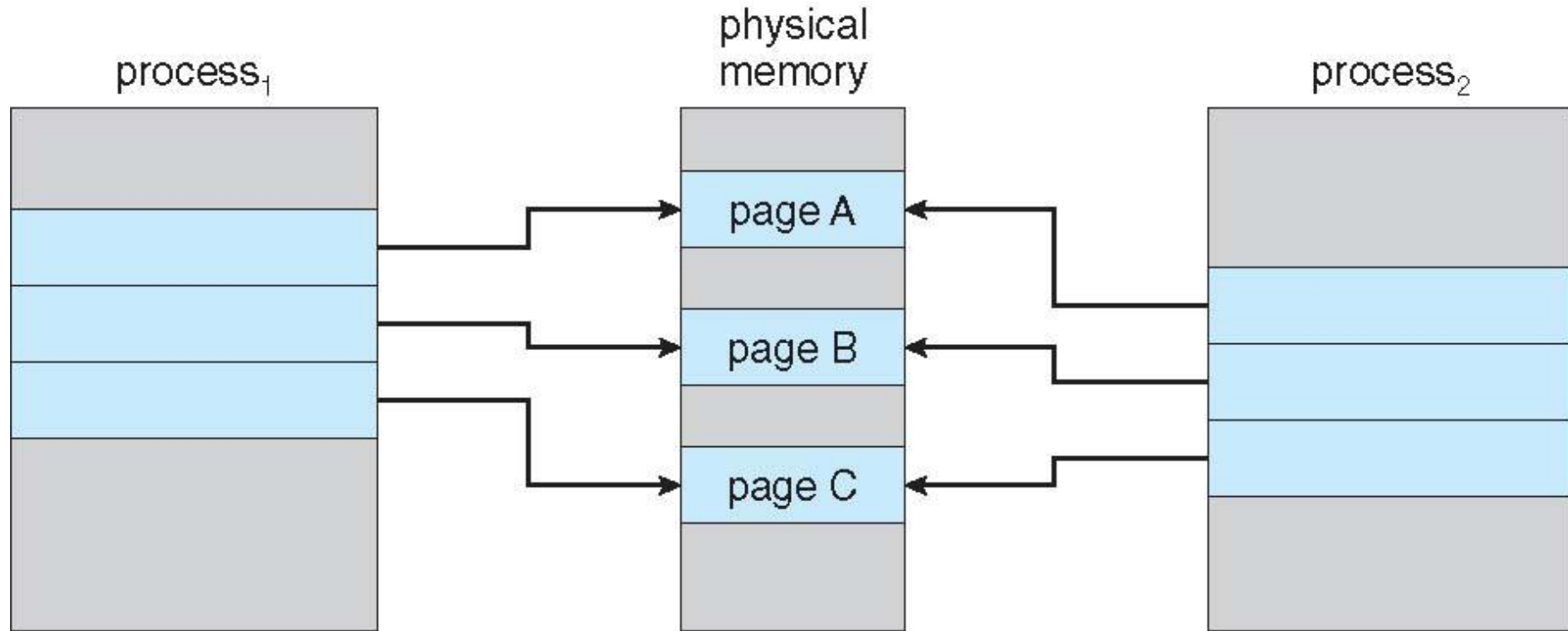
Demand Paging Optimization

- Swap space I/O faster than file system I/O even if on the same device
 - Swap allocated in larger chunks, less management needed than file system
- Copy entire process image to swap space at process load time
 - Then page in and out of swap space
 - Used in older BSD Unix
- Demand page in from program binary on disk, but discard rather than paging out when freeing frame
 - Used in Solaris and current BSD
 - Still need to write to swap space
 - Pages not associated with a file (like stack and heap) – **anonymous memory**
 - Pages modified in memory but not yet written back to the file system
- Mobile systems
 - Typically don't support swapping
 - Instead, demand page from file system and reclaim read-only pages (such as code)

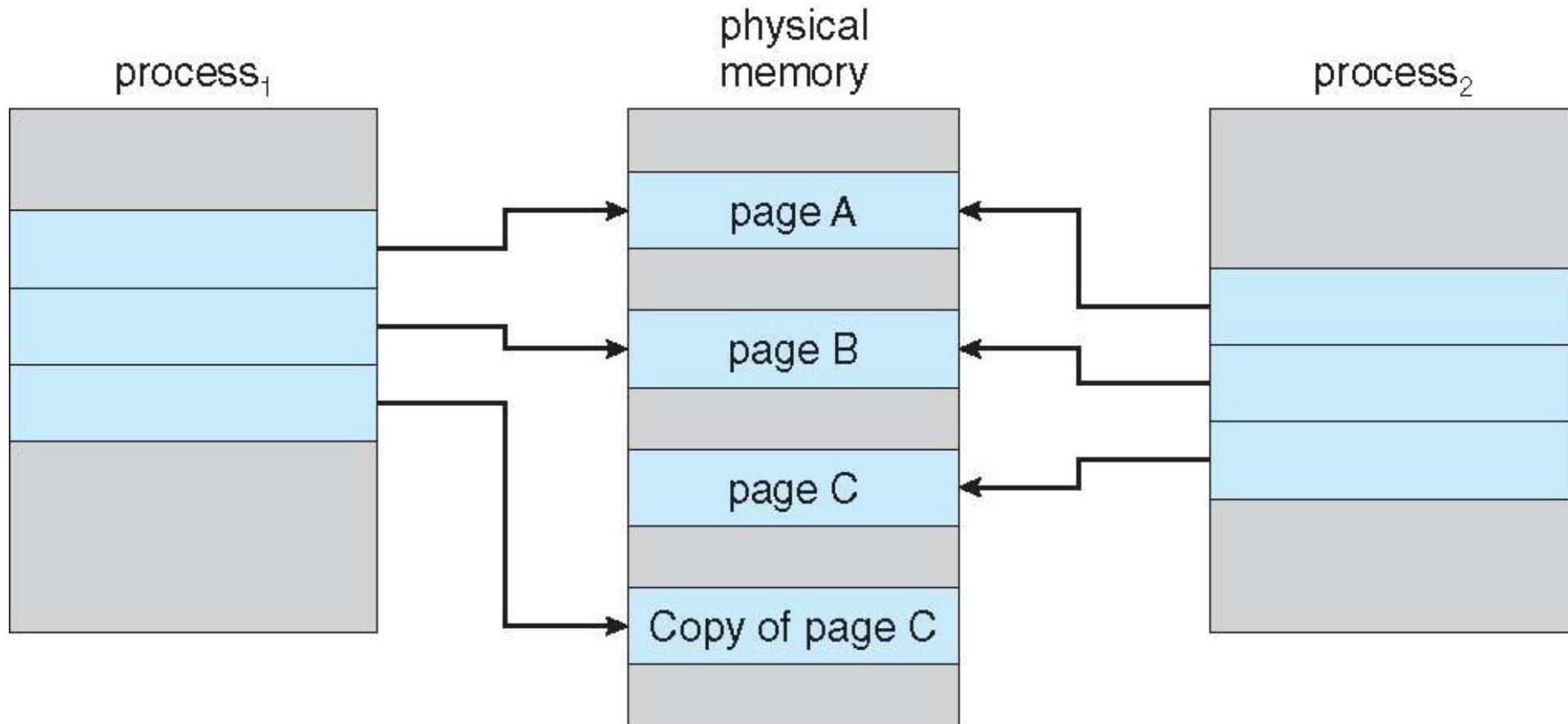
Copy on write

- **Copy-on-Write (COW)** allows both parent and child processes to initially *share* the same pages in memory
 - If either process modifies a shared page, only then is the page copied
- COW allows more efficient process creation as only modified pages are copied
- In general, free pages are allocated from a **pool of zero-fill-on-demand** pages
 - Pool should always have free frames for fast demand page execution
 - Don't want to have to free a frame as well as other processing on page fault
 - Why zero-out a page before allocating it?
- **vfork()** variation on fork() system call has parent suspend and child using copy-on-write address space of parent
 - Designed to have child call exec()
 - Very efficient

Before Process 1 Modified Page C



After Process 1 Modified Page C



What Happens if there is no free frame?

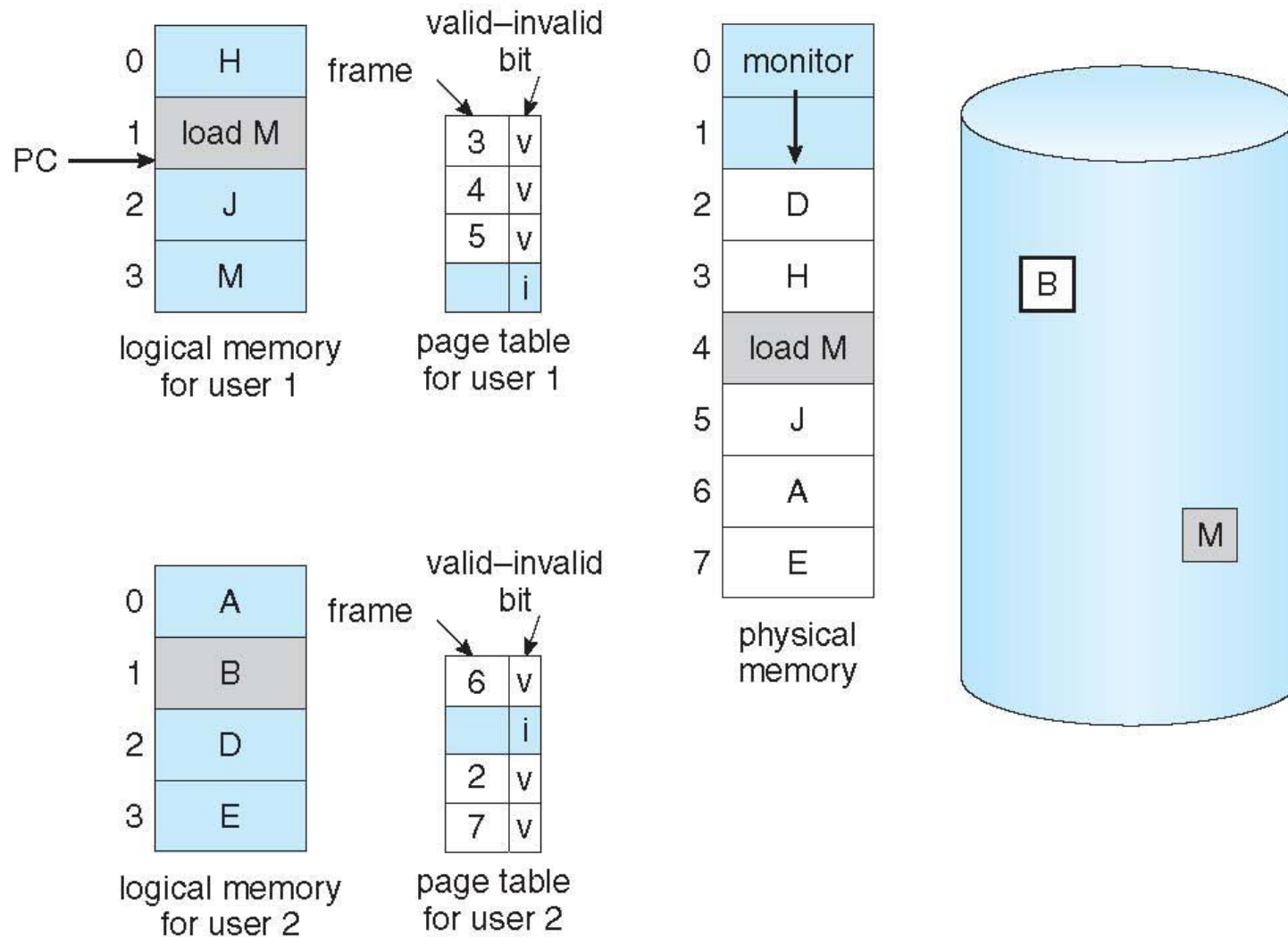
- Used up by process pages
- Also in demand from the kernel, I/O buffers, etc
- How much to allocate to each?
- Page replacement – find some page in memory, but not really in use, page it out
 - Algorithm – terminate? swap out? replace the page?
 - Performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

Page Replacement

A page replacement in an operating system is the process in which a page from the main memory is replaced with a page from the secondary memory.

- Prevent over-allocation of memory by modifying page-fault service routine to include page replacement
- Use modify (dirty) bit to reduce overhead of page transfers – only modified pages are written to disk
- Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory

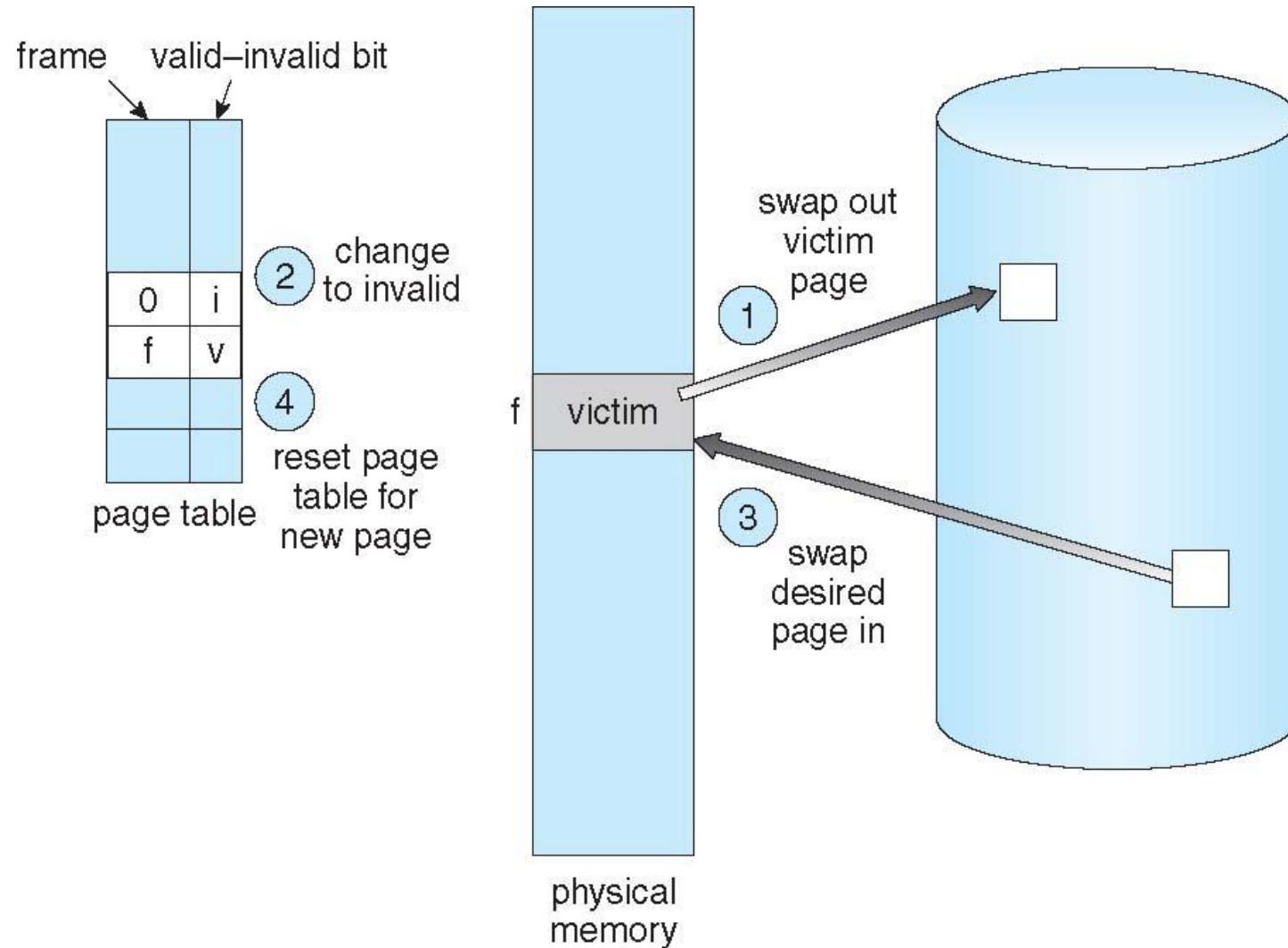
Need for Page Replacement



Basic Page Replacement

1. Find the location of the desired page on disk
 2. Find a free frame
 1. If there is a free frame, use it
 2. If there is no free frame, use a page replacement algorithm to select a victim frame
 3. Write victim frame to disk if dirty
 3. Bring the desired page into the (newly) free frame; update the page and frame tables
 4. Continue the process by restarting the instruction that caused the trap
- Note now potentially 2 page transfers for page fault – increasing EAT

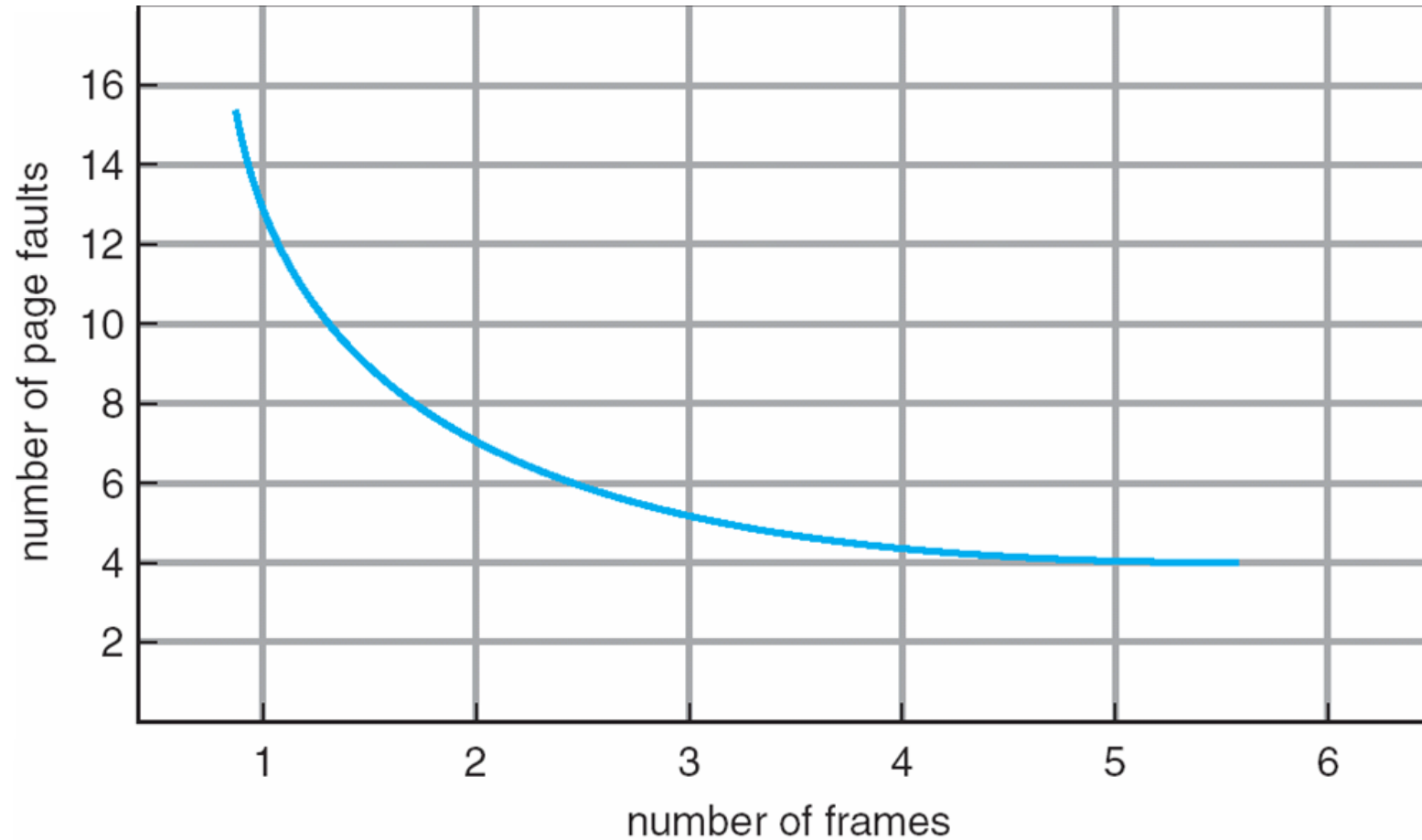
Page Replacement



Page and Frame Replacement Algorithm

- **Frame-allocation algorithm** determines
 - How many frames to give each process
 - Which frames to replace
- **Page-replacement algorithm**
 - Want lowest page-fault rate on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
 - String is just page numbers, not full addresses
 - Repeated access to the same page does not cause a page fault
 - Results depend on number of frames available

Graph of Page Faults Versus The Number of Frames



Types of Page Replacement Algorithm

- There are three types of page replacement algorithm
 1. FIFO Algorithm
 2. Optimal page replacement algorithm
 3. Least Recently used Algorithm

First in First out Algorithm

- This is the simplest page replacement algorithm.
- In this algorithm, the operating system keeps track of all pages in the memory in a queue, **the oldest page is in the front of the queue.**
- When a **page needs to be replaced page in the front of the queue is selected for removal.**

- **Lets take an example**

Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3 page frames.
Find the number of page faults.

Example FIFO

- Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3 page frames. Find the number of page faults.

	Bits Number						
	1	3	0	3	5	6	3
Page Frames			0	0	0	0	3
		3	3	3	3	6	6
	1	1	1	1	5	5	5
Page Fault	Miss	Miss	Miss	Hit	Miss	Miss	Miss

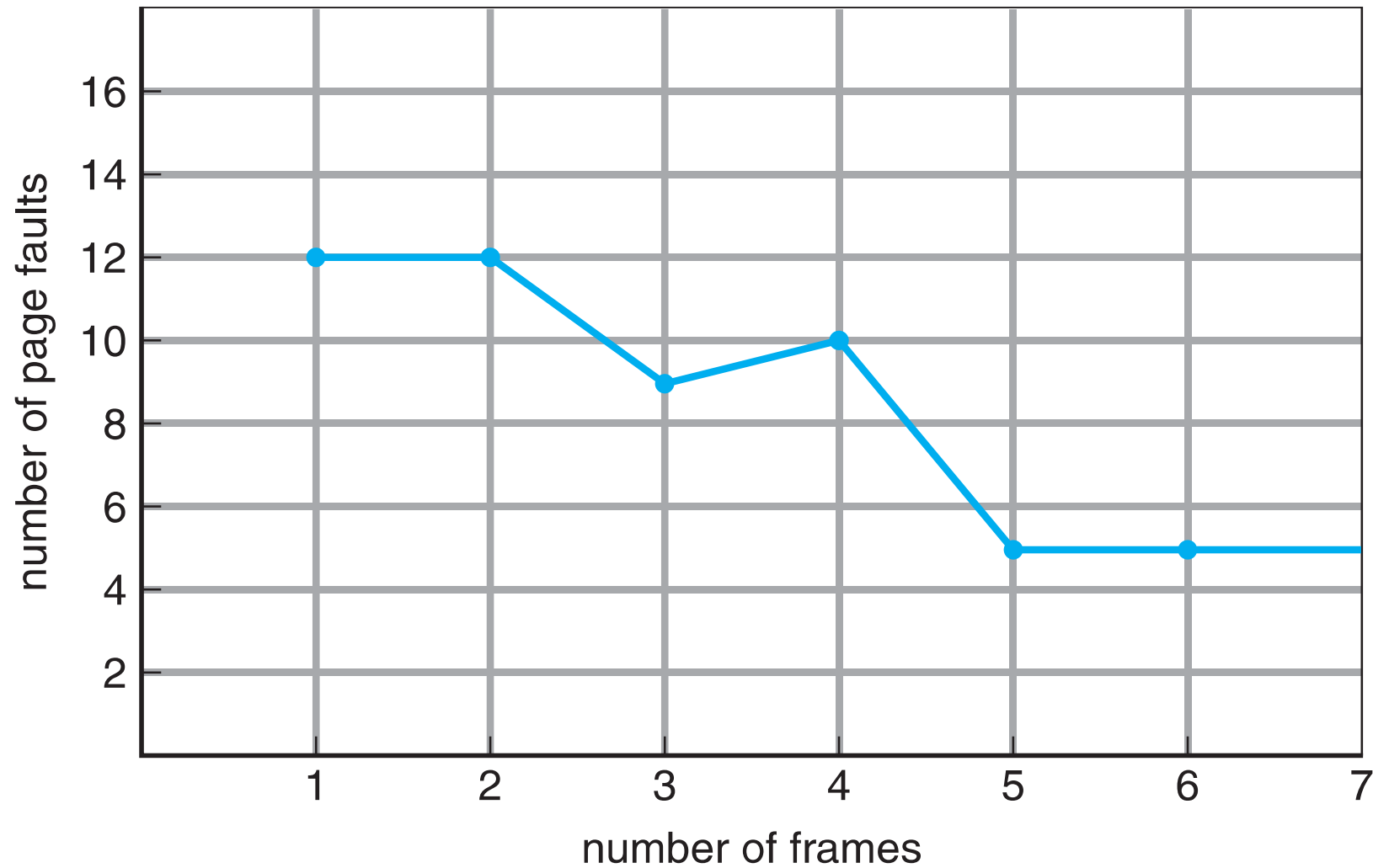
Total no of page faults = 6

FIFO Algorithm

- **Belady's Anomaly**

- While using the First In First Out algorithm, the number of page faults increases by increasing the number of frames. This

Belady Anomaly



Optimal Algorithm

- Replace page that will not be used for longest period of time
- Lets take an example
 - Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4 page frame. Find number of page fault.

Example:

- Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4 page frame. Find number of page fault.

	Bits Number													
	7	0	1	2	0	3	0	4	2	3	0	3	2	3
Page Frames				2	2	2	2	2	2	2	2	2	2	2
			1	1	1	1	1	4	4	4	4	4	4	4
		0	0	0	0	0	0	0	0	0	0	0	0	0
	7	7	7	7	7	3	3	3	3	3	3	3	3	3
Page Fault	Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total page fault = 6

LRU algorithm Performance

- Number of Page Hits = 8
- Number of Page Faults = 6

- The Page Hit Percentage
$$= \frac{\text{No of page hit}}{\text{Total No of pages}} \times 100$$

$$= \frac{8}{14} \times 100 = 57\%$$

LRU algorithm Performance

- Number of Page Hits = 8
- Number of Page Faults = 6
- The Page Hit Percentage

$$= \frac{\text{No of page fault}}{\text{Total No of pages}} \times 100$$

$$= \frac{6}{14} \times 100 \cong 42\%$$

LRU Algorithm

- Counter implementation
 - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
 - When a page needs to be changed, look at the counters to find smallest value
 - Search through table needed

LRU Algorithm

- Stack implementation
 - Keep a stack of page numbers in a double link form:
 - Page referenced:
 - move it to the top
 - requires 6 pointers to be changed
 - But each update more expensive
 - No search for replacement
- LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly

Example:

- Consider the reference string 6, 1, 1, 2, 0, 3, 4, 6, 0, 2, 1, 2, 1, 2, 0, 3, 2, 1, 2, 0 for a memory with three frames and calculate number of page faults by using Least Recently Used (LRU) Page replacement algorithms.

	Bits Number																			
	6	1	1	2	0	3	4	6	0	2	1	2	1	2	0	3	2	1	2	0
Page Frames				2	2	2	4	4	4	2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	3	3	3	0	0	0	0	0	0	0	0	0	1	1	1
	6	6	6	6	0	0	0	6	6	6	1	1	1	1	1	3	3	3	3	0
Page Fault	<u>Miss</u>	<u>Miss</u>	<u>Hit</u>	<u>Miss</u>	<u>Miss</u>	<u>Miss</u>	<u>Miss</u>	<u>Miss</u>	<u>Miss</u>	<u>Miss</u>	<u>Miss</u>	<u>Hit</u>	<u>Hit</u>	<u>Hit</u>	<u>Hit</u>	<u>Miss</u>	<u>Hit</u>	<u>Miss</u>	<u>hit</u>	<u>Miss</u>

- Total Page faults = 13

LRU algorithm Performance

- Number of Page Hits = 7
- Number of Page Faults = 13

- The Page Hit Percentage

$$= \frac{\text{No of page hit}}{\text{Total No of pages}} \times 100$$

$$= \frac{7}{20} \times 100 = 35\%$$

- The Page Fault Percentage = 100 - Page Hit Percentage = 100 - 35 = 65%

LRU algorithm Performance

- Number of Page Hits = 7
- Number of Page Faults = 13

- The Page Hit Percentage

$$= \frac{\text{No of page fault}}{\text{Total No of pages}} \times 100$$

$$= \frac{13}{20} \times 100 = 65\%$$